

COMPTE RENDU DE TP

Méthodes itératives et déformation d'un pont

Encadrant :
M. Antoine Tonnoir

Présenté par :
Mouad SHERADJ
DRISSI
Amr ZAKI SALIH

Année universitaire : 2024-2025

Remerciements

Nous tenons à remercier Monsieur Tonnoir pour son accompagnement, ainsi que pour cette opportunité qui nous a permis d'appliquer concrètement les concepts théoriques étudiés en classe. Ce travail pratique nous a aidé à comprendre l'importance et l'intérêt des méthodes numériques dans la résolution de problèmes complexes, et à renforcer nos compétences en programmation et en analyse mathématique.

Table des matières

1	Introduction	3
2	Déformation statique d'un pont	3
2.1	Méthode de splitting	6
2.2	Application numérique	8
3	Modes de vibration	19
3.1	Importance des modes propres	24
3.2	Tests et interprétations	26
3.3	Affinage et valeurs propres	28
4	Conclusion	29
1	Annexe	30

1 Introduction

Ce TP a pour objectif d'analyser l'intérêt des méthodes itératives dans la résolution de systèmes linéaires, en particulier dans le cadre de simulations de phénomènes physiques tels que des problématiques comme le calcul de la déformation d'une structure. Nous explorerons ainsi les différentes méthodes de descente de Gradient, Jacobi, Gauss-Seidel ainsi que Gradient Conjugué, en mettant en évidence que leur efficacité dépend du problème à résoudre. Nous envisagerons par ailleurs l'étude des modes propres et en quoi ces derniers sont extrêmement importants à prendre en compte dans la conception de structure. Cette étude s'inscrit dans une démarche de modélisation et d'optimisation de la résolution de systèmes linéaires, des enjeux cruciaux dans de nombreux domaines de l'ingénierie numérique.

2 Déformation statique d'un pont

Un pont est une structure permettant de franchir des obstacles. Nous choisissons de modéliser ce dernier par un réseau d'arcs(représentant les poutres) et de noeuds(représentant les jonctions). On notera ainsi $\underline{p}^i \in \mathbb{R}^3$ la position du noeud i et $\underline{D}^i \in \mathbb{R}^3$ son déplacement.

Par ailleurs, en appliquant le Principe Fondamental de la Dynamique(PFD), on montre que :

$$m_i \ddot{\underline{D}}_i = - \sum_j k_{ij} ((\underline{D}^i - \underline{D}^j) \cdot \underline{u}^{ij}) \underline{u}^{ij} \quad \text{avec} \quad \underline{u}^{ij} = \frac{\underline{p}^i - \underline{p}^j}{\|\underline{p}^i - \underline{p}^j\|_2} \in \mathbb{R}^3$$

où :

- $m_i \ddot{\underline{D}}_i$ représente l'accélération du noeud i multipliée par sa masse
- $-\sum_j k_{ij} ((\underline{D}^i - \underline{D}^j) \cdot \underline{u}^{ij}) \underline{u}^{ij}$ correspond à la somme des forces exercées par les ressorts connectées à i

Dans un premier temps, nous allons considérer qu'on se trouve dans un système stationnaire en temps, c'est à dire $\underline{D}_i(t) = \underline{d}_i$ ou $\underline{d}^i \in \mathbb{R}^3$.

L'équation obtenue par le PFD peut se réécrire sous forme matricielle comme un système linéaire $\mathbb{K} \underline{d} = \underline{F}$ où :

- $\mathbb{K}$ est la matrice de rigidité globale de taille $3n \times 3n$:

$$\mathbb{K} = \begin{pmatrix} K_{11} & \cdots & K_{1n} \\ \vdots & \ddots & \vdots \\ K_{n1} & \cdots & K_{nn} \end{pmatrix}, \quad \text{avec } K_{ij} \in \mathcal{M}_{3,3}(\mathbb{R}), \forall i, j.$$

— $\underline{d} \in \mathbb{R}^{3n}$ est le vecteur regroupant les d_i :

$$\mathbf{d} = \begin{pmatrix} d_1 \\ \vdots \\ d_n \end{pmatrix}, \quad \text{avec } d_i \in \mathbb{R}^3, \forall i.$$

— $\underline{F}$ est le vecteur des forces appliquées.

On sait que :

$$\mathbb{K}\underline{d} = \sum_{j=1}^n K_{ij} d_j$$

Donc on trouve que :

$$\sum_{j=1}^n K_{ij} d_j = - \sum_{j=1, j \neq i}^n k_{ij} ((\underline{d}^i - \underline{d}^j) \cdot \underline{u}_{ij}) \underline{u}_{ij}, \quad \forall i \in \{1, \dots, n\}$$

On simplifie cette somme en décomposant les contributions des déplacements d_i et d_j :

$$\begin{aligned} &= - \sum_{j=1, j \neq i}^n k_{ij} (\underline{d}^i \cdot \underline{u}_{ij}) \underline{u}_{ij} + \sum_{j=1, j \neq i}^n k_{ij} (\underline{d}^j \cdot \underline{u}_{ij}) \underline{u}_{ij} \\ &= - \sum_{j=1, j \neq i}^n k_{ij} \underline{u}_{ij} \underline{u}_{ij}^\top \underline{d}^i + \sum_{j=1, j \neq i}^n k_{ij} \underline{u}_{ij} \underline{u}_{ij}^\top \underline{d}^j \\ &= - \sum_{j=1, j \neq i}^n k_{ij} \underline{u}_{ij} \underline{u}_{ij}^\top \underline{d}^i + \sum_{j=1, j \neq i}^n \left(k_{ij} \underline{u}_{ij} \underline{u}_{ij}^\top \right) \underline{d}^j \end{aligned}$$

On remarque que la somme se décompose en termes liés aux déplacements. On a ainsi que le premier terme en d_i correspond à la somme des contributions de tous les arcs connectés à i :

$$\sum_j \left(k_{ij} \underline{u}_{ij} \underline{u}_{ij}^\top \right) \underline{d}^i$$

correspond donc au bloc diagonale K_{ii} de la matrice $\mathbb{K}$.

Et de même, le second terme inclut les contributions des déplacements des noeuds voisins j :

$$- \sum_j \left(k_{ij} \underline{u}_{ij} \underline{u}_{ij}^\top \right) \underline{d}^j$$

On obtient la structure de la matrice $\mathbb{K}$ qui est composée de blocs K_{ij} qui regroupent les contributions des forces entre les noeuds i et j . Ces blocs sont définis comme suit :

$$K_{ij} = \begin{cases} -k_{ij} \underline{u}_{ij} \cdot \underline{u}_{ij}^\top, & \text{si } i \neq j, \\ \sum_{j=1, j \neq i} k_{ij} \underline{u}_{ij} \cdot \underline{u}_{ij}^\top, & \text{si } i = j. \end{cases}$$

On remarque dès lors que la matrice $\mathbb{K}$ est symétrique car $u_{ij} u_{ij}^T$ est symétrique et $k_{ij} = k_{ji}$.

D'autre part :

$$\mathbf{d}^T \mathbb{K} \mathbf{d} = \sum_{i,j} \mathbf{d}_i^T \mathbf{K}_{ij} \mathbf{d}_j.$$

Si $i \neq j$, on a $\mathbf{K}_{ij} = -k_{ij} \mathbf{u}_{ij} \mathbf{u}_{ij}^T$, ce qui nous donne :

$$\mathbf{d}_i^T \mathbf{K}_{ij} \mathbf{d}_j = -k_{ij} (\mathbf{d}_i^T \mathbf{u}_{ij}) (\mathbf{d}_j^T \mathbf{u}_{ij}).$$

$$= k_{ij} \sum_{i \neq j} (\mathbf{d}_i^T \mathbf{u}_{ij})^2.$$

Si $i = j$, on a :

$$\mathbf{d}_i^T \mathbf{K}_{ii} \mathbf{d}_i = \sum_{j \neq i} k_{ij} (\mathbf{d}_i^T \mathbf{u}_{ij})^2.$$

Donc, on trouve que :

$$\mathbf{d}^T \mathbf{K} \mathbf{d} = \sum_{i,j} k_{ij} (\mathbf{d}^T \mathbf{u}_{ij})^2.$$

Or puisque $\mathbf{K}_{ii} \geq 0$ et $(\mathbf{d}^T \mathbf{u}_{ij})^2 \geq 0$. D'où $\mathbb{K}$ est semi-défini positif.

Passons désormais au **cas simple** d'une jonction entre $\underline{p}^1 = [0, 0, 0]$ et $\underline{p}^2 = [1, 0, 0]$. On sait que la matrice $\mathbb{K}$ est définie de la sorte :

$$\mathbb{K} = \begin{bmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{bmatrix}$$

Calculons désormais les différentes sous-matrices de cette matrice :

- $K_{11} = k_{12} u_{12} u_{12}^T$
- $K_{12} = -k_{12} u_{12} u_{12}^T$
- $K_{21} = -k_{21} u_{21} u_{21}^T$

— $K_{22} = k_{12}u_{12}u_{12}^T$

et on a que :

$$u_{12} = \frac{p_2 - p_1}{\|p_2 - p_1\|} = [1, 0, 0], \quad \text{puisque } \|p_2 - p_1\| = 1.$$

$$\text{En utilisant } u_{12} = [1, 0, 0], \quad u_{12}u_{12}^T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

et :

$$u_{21} = \frac{p_1 - p_2}{\|p_1 - p_2\|} = [-1, 0, 0], \quad \text{puisque } \|p_1 - p_2\| = 1.$$

$$\text{En utilisant } u_{21} = [-1, 0, 0], \quad u_{21}u_{21}^T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

On obtient donc ainsi la matrice :

$$\mathbb{K} = \begin{bmatrix} k_{12} & 0 & 0 & -k_{12} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ -k_{12} & 0 & 0 & k_{12} & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}.$$

avec $k_{12} = k_{21}$

2.1 Méthode de splitting

Une méthode de résolution du système $Ax = b$, où A appartient à $M_n(\mathbb{R})$, inversible et b appartient à $\mathbb{R}^n$, est dite itérative si elle converge vers la solution x après un nombre infini d'itérations. Elle repose sur la construction d'une suite convergente de vecteurs x^k telle que :

$$x = \lim_{k \rightarrow \infty} x^k,$$

où x est la solution recherchée. Autrement dit, les méthodes itératives visent à obtenir une solution approchée.

On pose $A = M - N$, avec M une matrice facile à inverser. Une méthodes de splitting est une méthode itérative qui consiste à calculer une suite de solutions approchées x^k

en utilisant la relation suivante :

$$Mx^{k+1} = Nx^k + b$$

Pour que cette méthode converge, il est nécessaire que le rayon spectral de $M^{-1}N$ soit strictement inférieur à 1, soit $\rho(M^{-1}N) < 1$.

Il existe différentes méthodes de splitting pour la résolution d'un système linéaire. Montrons que la méthode de relaxation en fait partie.

Partant de la définition, on a la relation suivante :

$$\underline{x}^{k+1} = (1 - \theta)\underline{x}^k + \theta\tilde{\underline{x}}^k$$

On sait par ailleurs que $\tilde{\underline{x}}^k$ est une solution calculée avec une méthode de splitting, donc :

$$\tilde{\underline{x}}^k = M^{-1}N\underline{x}^{k-1} + M^{-1}\underline{b}$$

On remplace la relation de $\tilde{\underline{x}}^k$ dans l'itération donnée, ce qui donne :

$$\underline{x}^{k+1} = (1 - \theta)\underline{x}^k + \theta(M^{-1}N\underline{x}^k + M^{-1}\underline{b}).$$

En réorganisant les termes, on obtient :

$$\underline{x}^{k+1} = ((1 - \theta) + \theta M^{-1}N) \underline{x}^k + \theta M^{-1}\underline{b}.$$

En multipliant par $\frac{M}{\theta}$, on transforme cette équation dans une forme semblable à celle de la méthode de splitting :

$$\frac{M}{\theta} \underline{x}^{k+1} = \left(\frac{1 - \theta}{\theta} M + N \right) \underline{x}^k + \underline{b}.$$

Par identification avec la relation de la méthode de splitting :

$$M' \underline{x}^{k+1} = N' \underline{x}^k + \underline{b},$$

on déduit que les matrices M' et N' pour la méthode de relaxation sont exprimées en

fonction des matrices M et N de la méthode de splitting.

$$M' = \frac{M}{\theta},$$

et

$$N' = \frac{1 - \theta}{\theta} M + N.$$

2.2 Application numérique

Dans notre contexte, les méthodes de splitting sont particulièrement pratiques pour résoudre des problèmes liés à la mécanique. Nous nous permettons ainsi, dans un premier temps, d'étudier un pont de base qui n'a pas été créé par nos soins, afin de pouvoir déterminer l'efficacité primaire des différentes méthodes.

2.2.1 Gradient

On se donne comme critère de convergence que $\forall x$ si et seulement si la suite $(\underline{x})_k^k$ converge vers la même limite $\underline{x}^* = \underline{x}$ alors la méthode itérative est convergente. Nous avons dès lors les paramètres suivants :

- $tol = 1e - 5$
- $itemax = 1000$
- $relax = 1$

Ces paramètres sont les paramètres de bases que l'on va faire varier afin de déterminer l'impact qu'ils ont sur les différentes méthodes. Ainsi, regardons pour la méthode de Gradient ce que ces paramètres donnent pour une pression uniforme :

Pour $tol = 1e - 5, itemax = 1000, relax = 1$:

On remarque que la méthode de Gradient prend 866 itérations pour converger vers une solution avec pour erreur relative de l'ordre de $err = 9.903588961787733e - 06$.

Pour $tol = 1e - 8, itemax = 1000, relax = 1$:

On remarque que dès lors, la méthode de Gradient ne semble pas converger. Il n'y a pas assez d'itérations pour pouvoir atteindre une erreur relative inférieur à la tolérance $tol = 1e - 8$

Essayons avec un plus d'itérations $itemax = 2000$:

La méthode de Gradient semble converger vers une solution avec une erreur relative de l'ordre de $err = 9.968328725995475e - 09$ avec 1388 itérations.

Pour $tol = 1e - 3, itemax = 1000, relax = 1$:

La méthode converge vers une solution avec une erreur relative de $err = 0.0009860663439297036$

en 518 itérations.

Nous pouvons ainsi dire que la tolérance tol joue un rôle central dans la précision et la performance de la méthode numérique du Gradient. Une tolérance plus stricte demande un plus grand nombre d'itérations pour atteindre une solution, augmentant ainsi le temps de calcul mais aussi donc la précision. Ajuster la tolérance permet donc de trouver un compromis entre la précision et les ressources de calculs disponibles.

Faisons maintenant varier la relaxation :

Pour $tol = 1e - 5$, $itemax = 1000$, $relax = 0.5$:

On remarque qu'il y a convergence vers une solution avec une erreur relative de $err = 9.97869496094176e - 06$ en 274 itérations, cependant, le processus de convergence semble plus chaotique(cf courbe).

Pour $tol = 1e - 5$, $itemax = 1000$, $relax = 1.5$:

On remarque que le nombre d'itérations est trop petit pour qu'il y ait convergence,puisque pour 1000 itérations, on a une erreur relative $err = 2.4380604144966347e - 05$. Essayons avec un plus grand nombre d'itérations $itemax = 2000$:

On remarque que la méthode converge en 1082 itérations vers une solution avec une erreur relative $err = 9.939032318794113e - 06$ inférieure à la tolérance $tol = 1e - 5$. La courbe représentant le processus de convergence semble par ailleurs très linéaire.

Le facteur de relaxation $relax$ a un impact significatif sur le comportement et l'efficacité de la méthode. Avec une relaxation plus basse, la méthode converge plus vite, mais le processus de convergence est bien plus chaotique, affirmant que la stabilité de la convergence n'est pas bonne. Tandis qu'avec une relaxation plus forte, la convergence est permise avec une courbe beaucoup plus linéaire, indiquant un comportement plus stable. Cependant, une telle nécessite un nombre d'itérations plus élevé pour atteindre la tolérance.

2.2.2 Jacobi

La méthode de Jacobi est une méthode de splitting avec pour choix défini $M = D = \text{Diag}(A)$ et $N = D - A$. Elle est très simple d'implémentation puisque les calculs x_i^{k+1} n'utilisent que les valeurs de x^k , sans dépendance circulaire entre les composantes. Cependant, cette dernière se heurte à quelques limites. En effet, la méthode de Jacobi peut être lente à converger si A n'est pas bien conditionnée ou si elle n'est pas à diagonale dominante. Tout comme la méthode du Gradient, nous allons faire varier les différents paramètres suivants :

- $tol = 1e - 5$
- $itemax = 1000$
- $relax = 1$

Ainsi, on a que :

Pour $tol = 1e - 5, itemax = 1000, relax = 1$:

La méthode converge en 456 itérations avec une erreur relative $err = 9.955174688742798e - 06$.

Pour $tol = 1e - 8, itemax = 1000, relax = 1$:

La méthode converge en 731 itérations avec une erreur relative $err = 9.767832886022135e - 09$

Pour $tol = 1e - 3, itemax = 1000, relax = 1$:

La méthode converge en 273 itérations avec une erreur relative 0.000999776615907182

On remarque ainsi que tout comme pour la méthode du Gradient, la tolérance tol joue un rôle important dans la précision et la performance de la méthode numérique. Cependant, la méthode de Jacobi semble plus efficace en termes de rapidité que la méthode du Gradient, puisque pour des erreurs relatives presque égales et les mêmes tolérances, on a que le nombre d'itérations est inférieur ($tol = 1e - 8$, 731 itérations et avec une erreur relative $err = 9.767832886022135e - 09$ pour Jacobi et $err = 9.968328725995475e - 09$ avec 1388 itérations pour gradient).

Faisons maintenant varier la relaxation :

Pour $tol = 1e - 5, itemax = 1000, relax = 0.5$:

La méthode converge en 917 itérations avec une erreur relative $err = 9.92612277913739e - 06$. Par ailleurs, le processus de convergence semble bien se passer puisque la courbe est linéaire.

Pour $tol = 1e - 5, itemax = 1000, relax = 1.5$:

La méthode diverge, on a une erreur relative $err = \infty$.

La divergence de la méthode semble aux premiers abords intrigante. En effet, on peut se demander pourquoi pour la méthode du Gradient il n'y a pas divergence et tout autant pourquoi quand il y a convergence, la méthode du Gradient est plus rapide bien que moins stable. En fait, la relaxation détermine l'ampleur avec laquelle on ajuste les valeurs de x à chaque itération. Si la relaxation est trop forte, les mises à jour des valeurs de x deviennent trop grandes ce qui peut rendre l'algorithme instable. Avec un facteur de relaxation élevé donc, les valeurs peuvent osciller au lieu de se rapprocher progressivement

de la solution. Par ailleurs, pour la méthode de Jacobi, la convergence est liée à la norme du rayon spectrale de la matrice associée ; si ce rayon est supérieur à 1, la méthode divergera forcément. La méthode de Jacobi bien que plus efficace en certains termes que la méthode du Gradient reste néanmoins très sensible à un mauvais choix de relaxation. Il est donc essentiel de choisir un paramètre adapté pour garantir la convergence tout en optimisant le nombre d'itérations.

2.2.3 Gauss-Seidel

La méthode de Gauss-Seidel est tout comme Jacobi une méthode de splitting avec quant à elle pour choix défini $M = D - E$ et $N = M - A$ avec E matrice triangulaire inférieure telle que $E_{ij} = -A_{ij}$ si $i > j$. Elle est souvent considérée comme une amélioration de la méthode de Jacobi, car elle utilise immédiatement les nouvelles valeurs calculées à chaque itération ce qui accélère généralement la convergence. Cependant, comme pour Jacobi, le fait que la matrice ne soit pas diagonalement dominante ou symétrique définie positive n'assure pas la convergence. Par ailleurs, de manière opposée à Jacobi où chaque itération peut être calculée en parallèle, Gauss-Seidel nécessite que chaque élément de x soit mis à jour pour chaque séquence.

Tout comme pour les autres méthodes, faisons varier dans un premier temps la tolérance tol :

Pour $tol = 1e - 5, itemax = 1000, relax = 1$:

La méthode converge en 235 itérations avec une erreur relative $err = 9.987070629663277e - 06$.

Pour $tol = 1e - 8, itemax = 1000, relax = 1$:

La méthode converge en 372 itérations avec une erreur relative $err = 9.98805470060722e - 09$

Pour $tol = 1e - 3, itemax = 1000, relax = 1$:

La méthode converge en 144 itérations avec une erreur relative 0.0009819977139200488

On remarque que la méthode de Gauss-Seidel semble converger bien plus vite que la méthode de Jacobi ainsi que du Gradient. De plus, l'erreur relative obtenue pour des valeurs de tolérances beaucoup plus strictes est très proche de la tolérance ce qui témoigne de l'efficacité et de la stabilité de cette méthode.

Faisons désormais varier la relaxation :

Pour $tol = 1e - 5, itemax = 1000, relax = 0.5$:

La méthode converge en 475 itérations avec une erreur relative $err = 9.980586508091437e-06$. Par ailleurs, le processus de convergence semble bien se passer puisque la courbe est linéaire.

Pour $tol = 1e-5, itemax = 1000, relax = 1.5$:

La méthode converge en 155 itérations avec une erreur relative $err = 9.98042408291697e-06$.

Pour $tol = 1e-5, itemax = 1000, relax = 2.0$:

La méthode diverge. On a une erreur qui ne cesse d'augmenter.

La différence dans le comportement de divergence entre les méthodes de Jacobi et de Gauss-Seidel face à des facteurs de relaxations élevés est liée à leur fonctionnement interne et l'utilisation de nouvelles valeurs lors des itérations. A chaque itération, les nouvelles valeurs calculées sont utilisées dans le calcul des composantes suivantes pour la méthode de Gauss-Seidel. Cela permet donc à cette méthode d'utiliser des informations plus récentes pour ajuster les calculs, ce qui stabilise l'algorithme et compense les effets d'une relaxation supérieure à 1. Quant à Jacobi, chaque itération utilise uniquement les valeurs de la solution précédente pour mettre à jour toutes les composantes de x . Les erreurs introduites à chaque étapes vont donc s'accumuler plus facilement, car les ajustements ne tiennent pas compte de nouvelles informations avec une relaxation supérieure à 1. C'est pourquoi le facteur de la relaxation influence plus fortement la méthode de Jacobi que celle de Gauss-Seidel, cependant, il est tout de même essentiel de choisir une relaxation adaptée pour équilibrer vitesse de convergence et stabilité pour les deux méthodes et d'éviter de mettre des facteurs de relaxations trop élevés pour éviter de causer une instabilité.

2.2.4 Gradient Conjugué

La méthode du Gradient Conjugué est une méthode itérative qui permet de résoudre des systèmes linéaires, mais elle permet tout autant de minimiser des fonctions quadratiques. Elle est par ailleurs particulièrement efficace pour des systèmes de grande taille ou pour des matrices creuses. Elle s'appuie sur la minimisation de l'erreur en choisissant des directions de descente qui sont conjugués par rapport à la matrice. Cela permet donc de réduire l'erreur en un petit nombre d'itérations. Cependant, elle est très sensible au conditionnement ; si le conditionnement est trop élevé, la convergence peut être très lente. Par ailleurs, elle ne peut être appliquée directement que si A est symétrique définie positive.

Passons désormais à la variation des différentes variables :

Pour $tol = 1e-3, itemax = 1000, relax = 1$:

La méthode converge en 7 itérations avec une erreur relative $err = 3.715461184475011e - 15$. On remarque que l'erreur relative est extrêmement basse, testons ainsi avec une tolérance beaucoup plus petite.

Pour $tol = 1e - 18, itemax = 1000, relax = 1$:

La méthode converge en 21 itérations avec une erreur relative $err = 2.756133885434438e - 26$. La courbe forme comme un escalier, montrant qu'il y a certaines itérations où la méthode cherche le bon conjugué.

La méthode du gradient conjugué se distingue nettement des autres méthodes par sa rapidité et sa précision. Contrairement à Jacobi et Gauss-Seidel qui nécessitent un grand nombre d'itérations, la méthode du Gradient Conjugué converge en très peu d'itérations, même pour des tolérances très faibles. Elle surpasse la méthode du gradient simple en évitant les oscillations et les ralentissements qui peuvent survenir lors de la descente du gradient classique. La performance beaucoup plus importante de la méthode de descente de Gradient Conjugué peut s'expliquer dans un premier temps par le fait que pour Jacobi et Gauss-Seidel améliorent la solution de manière locale à chaque itération tandis que le Gradient Conjugué quant à lui utilise des directions optimales pour minimiser directement l'erreur globale. De plus, là où un paramètre de relaxation doit être ajusté pour garantir la convergence pour Gauss-Seidel et Jacobi, le gradient conjugué n'en a pas besoin ce qui simplifie son application. Aussi, les autres méthodes sont beaucoup plus sensibles au conditionnement.

2.2.5 Comparaison des différentes méthodes

Jusqu'ici, nous avons uniquement étudié l'influence des différents paramètres sur chacune des méthodes, appliquées à un pont simple avec une seule déformation. Nous allons désormais utiliser les paramètres qui vont suivre pour dans un premier temps comparer efficacement les différentes méthodes pour la pression uniforme puis le cisaillement. On regardera par ailleurs ce qu'il se passe avec une relaxation supérieure à 1, puis nous tenterons d'appliquer les méthodes sur un pont censé représenter le Sydney Harbour Bridge et un pont à treillis inversé.

- $tol = 1e - 8$
- $iteMax = 2000$
- $relax = 1.0$

On remarque dès lors que la méthode de **Gradient Conjugué** est bien plus efficace que les autres, elle converge en effet beaucoup plus vite pour la pression uniforme. Passons désormais au **cisaillement** avec toujours les mêmes paramètres. Même conclusion et même

nombres d'itérations. On essaye désormais avec une relaxation $relax = 1.5 > 1$ pour voir comment les méthodes se comportent. On essaye avec une relaxation $relax = 0.5$ et ici, on remarque que la méthode du Gradient est plus efficace que Jacobi ainsi que Gauss-Seidel. Cependant, elle semble plus oscillée et moins stable. Cela peut s'expliquer par le fait qu'à chaque itération, la méthode du Gradient suit la pente la plus raide pour réduire l'erreur. Une relaxation plus petite peut donc signifier des pas plus courts dans la direction de la pente, c'est ce qu'on remarque avec les oscillations sur le graphe. Le Gradient, puisque conçu pour exploiter les descentes directionnelles peut donc surpasser Gauss-Seidel et Jacobi. Comme dit précédemment, la méthode de Gradient Conjugué en plus de ne pas dépendre de la relaxation offre une convergence beaucoup plus rapide de part le fait que dans un premier temps, elle n'améliore pas la solution de manière locale à chaque itération mais plutôt utilise des directions conjuguées pour minimiser l'erreur. Par ailleurs, de manière théorique le Gradient Conjugué converge en un nombre d'itérations égal au rang de la matrice, ce qui n'est pas le cas ici, puisqu'on a un rang égal à 35 pour le pont simple mais qui prouve que la méthode de Gradient Conjugué est très efficace contrairement aux autres méthodes qui sont beaucoup plus sensibles au conditionnement.

Nous allons désormais passer à la comparaison des différentes méthodes pour le Sydney Harbour Bridge. Nous l'avons représenté de telle sorte à ce qu'il ressemble le plus possible au véritable, c'est le pont le plus large au monde (48,8 mètres) et ayant la plus haute arche en acier avec le sommet du pont situé à 134 mètres au-dessus des eaux. Pour cela, nous nous sommes appuyés sur un schéma avec une vue de côté du pont et nous avons essayer le plus possible de respecter les proportions ; 1 unité sur la modélisation correspond à 5.8m dans la vraie vie. Bien évidemment, les proportions ne sont pas tout à fait précises (intervalle de confiance $\pm 1m$ pour une unité). La matrice du pont est donc très grande et les temps de calculs sont donc très longs. Nous allons cependant essayer de tester les différentes méthodes sur le pont pour voir si ces dernières sont bien efficaces. Il faut cependant spécifier que le pont bien que semblant visuellement satisfaisant reflète très peu la réalité puisque, par exemple, les raideurs sont toutes à 1.0, alors que la réalité est sûrement bien plus éloignée. De même tout ce qui est masse ainsi que fixation ne correspondent sûrement pas aux données réelles du Sydney Harbour Bridge. On se propose ainsi d'étudier les convergences des différentes méthodes avec différents jeux de paramètres :

Pour $tol = 1e - 5, itemax = 1000, relax = 1.0$:

On remarque dès lors que les méthodes prennent énormément de temps pour converger et que la tolérance semble bien trop petite pour atteindre une convergence pour les méthodes qui convergent. On a que la méthode de Jacobi et du Gradient Conjugué diverge, et on

remarque que pour la méthode de Gauss-Seidel ainsi que du Gradient semble converger mais de manière très lente (on ne dépasse pas les 1, cependant, l'erreur relative décroît). Essayons dès lors avec un autre jeu de données.

Pour $tol = 1e - 5, itemax = 1000, relax = 0.5$:

Cette fois-ci la méthode de Jacobi semble converger mais comme pour l'autre jeu de donné, elle prend énormément de temps pour converger. La méthode de Gauss-Seidel converge tout autant, mais cette fois-ci la méthode du Gradient diverge. La relaxation comme exposé précédemment joue un rôle très important dans la convergence des méthodes ainsi que leur stabilité. On remarque en effet que la stabilité de la méthode du Gradient est très mauvaise avec cette relaxation alors que dans le jeu de donné précédent, elle était plutôt bonne. Essayons de l'augmenter un peu plus :

Pour $tol = 1e - 5, itemax = 1000, relax = 0.6$:

Il est difficile de déterminer une relaxation qui permettrait à toutes les méthodes de converger, cependant, bien que l'erreur relative soit extrêmement loin de la tolérance, une erreur relative assez basse est atteinte, semblant diminuer pour Jacobi et Gauss-Seidel mais qui semble augmenter pour la méthode du Gradient. D'autre part, avec une telle relaxation, on trouve que la convergence est très peu stable, ce qui n'est pas optimal. Il pourrait être intéressant d'augmenter le nombre d'itérations lorsque l'on aura trouvé la meilleure relaxation qui permettra de faire converger un maximum de méthodes.

Pour $tol = 1e - 5, itemax = 1000, relax = 1.5$:

La méthode du gradient semble beaucoup mieux marcher avec une relaxation plus importante. Cependant, Jacobi, puisque la relaxation est supérieure à 1, divergera forcément. On remarque qu'importe le jeu de donné, le Gradient Conjugué ne marche pas.

La matrice $\mathbb{K}$ a un conditionnement de $6.48e17$, ce qui est extrêmement mauvais et indique que la matrice est pratiquement singulière. De ce fait, la méthode du Gradient Conjugué est "bloquée" dans des itérations lentes. Une solution possible serait peut-être d'ajouter un préconditionneur pour réduire la dispersion des valeurs propres ou bien de changer la matrice pour pouvoir avoir un conditionnement beaucoup plus bas.

Nous pouvons ainsi exposer le fait que les facteurs telles que la relaxation ou encore le conditionnement jouent un rôle primordial dans la résolution de système linéaire avec méthodes itératives. En effet, la matrice $\mathbb{K}$, comme dit précédemment, présente un conditionnement extrêmement mauvais ce qui compromet la précision et l'efficacité des méthodes numériques. Par ailleurs, la relaxation joue un rôle déterminant dans la convergence et la stabilité des méthodes ; nous constatons qu'une relaxation trop faible

rend considérablement moins stable les méthodes comme celle du Gradient et une relaxation trop élevée peut causer la divergence de Jacobi. Cependant, cette matrice mal conditionnée nous a permis de mettre en exergue quelques points clés sur les méthodes itératives :

- La méthode de Jacobi converge lentement pour certains paramètres mais devient instable dès que la relaxation est supérieure à 1.0
- La méthode de Gauss-Seidel semble être un "passe-partout", elle montre en effet une robustesse remarquable dans la convergence bien que lente dans ces différents jeux de données
- La convergence de la méthode du Gradient dépend plus que fortement de la relaxation montrant une stabilité avec des relaxations supérieure à 1.0
- Le Gradient Conjugué, si la matrice est mal conditionnée peut ne pas converger du tout et bien que très efficace dans d'autres contextes peut tout autant ne pas l'être du tout.

Après avoir réalisé des tests sur le pont de Sydney Harbour Bridge, nous avons constaté que le conditionnement de la matrice $\mathbb{K}$ était trop élevé, ce qui perturbait la convergence des méthodes itératives utilisées. Dans cette perspective, nous avons conçu un pont plus simple, avec un nombre réduit de nœuds et d'arcs. Ce nouveau pont correspond à un modèle de pont à treillis inversé.



FIGURE 1.1 – Pont à treillis avec sa modélisation

Nous procédons à la comparaison des différentes méthodes itératives pour une valeur de relaxation variant entre 0 et 2. Les tests ont été réalisés dans le cas d'un cisaillement, avec les paramètres suivants :

$$\text{tol} = 10^{-5}, \quad \text{itemax} = 10^3.$$

Les résultats obtenus sont représentés sous forme de courbe montrant l'évolution de l'erreur relative finale (ou du dernier résidu) pour les différentes méthodes itératives. On évalue le $\log_{10}$ des erreurs relatives finales en fonction de la **relaxation**.

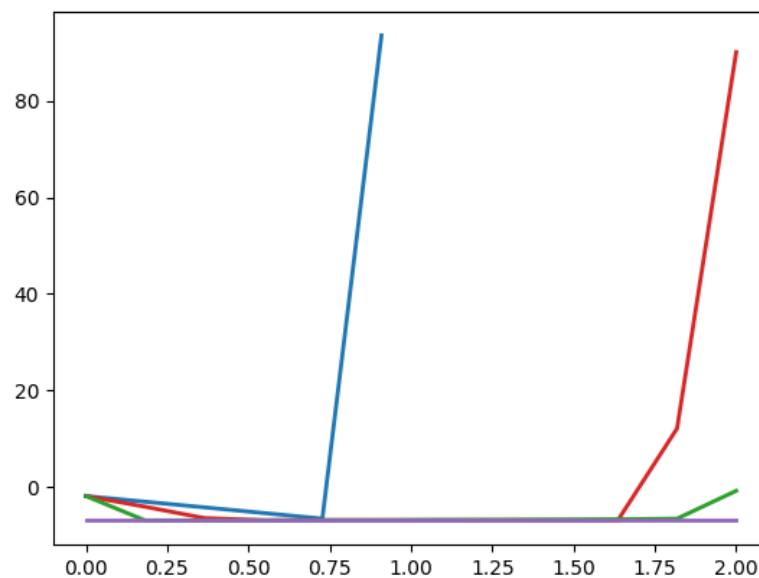


FIGURE 1.2

Nous avons également testé les performances pour une valeur fixe de $relax = 0.6$, en conservant les mêmes paramètres ($\text{tol} = 10^{-5}$ et $\text{itemax} = 10^3$).

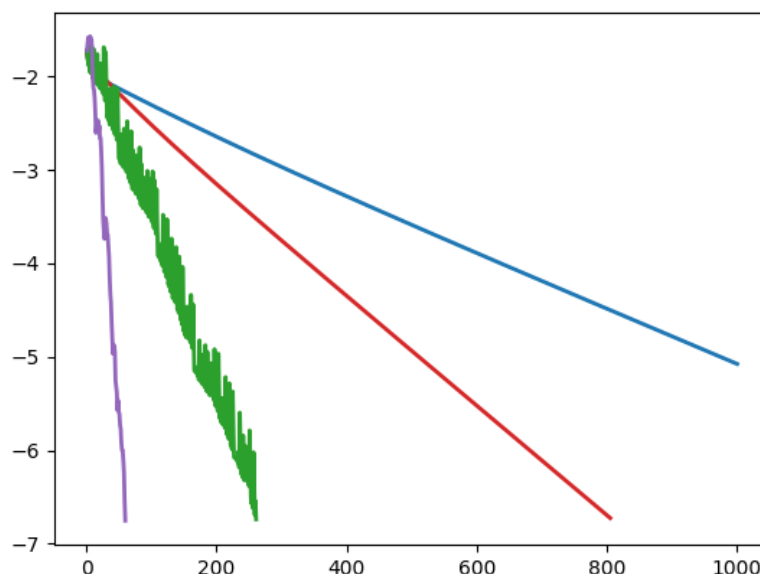


FIGURE 1.3

Les figures ci-dessus nous permettent d'analyser les performances des différentes méthodes itératives en fonction des paramètres choisis. La Figure 1.2 représente l'impact de la relaxation, qui varie entre 0 et 2, sur l'erreur relative finale des méthodes itératives Jacobi, Gauss-Seidel, Gradient Conjugué et Gradient. On remarque que la méthode Jacobi diverge dès qu'on dépasse 0,735 pour la relaxation, tandis que les autres méthodes restent stables, sauf pour la méthode Gauss-Seidel qui diverge vers 1,632. On constate qu'il y a une plage de relax qui permet une convergence pour toutes les méthodes : entre 0 et 0,7. D'autre part, la méthode Gradient semble tolérante aux variations de relax, sauf au début et à la fin où elle semble sensible, tandis que le Gradient Conjugué présente un comportement stable pour toutes les valeurs de relax, comme constaté auparavant pour le pont simple.

La Figure 1.3 se concentre sur les performances des méthodes pour une valeur fixe de $relax = 0.6$. On observe une décroissance progressive et régulière de l'erreur logarithmique pour toutes les méthodes traduisant une convergence. La méthode du Gradient Conjugué atteint la tolérance fixée (10^{-5}) plus rapidement que les autres, indiquant une meilleure efficacité. La méthode de Jacobi converge plus lentement, tandis que Gauss-seidel présente une décroissance plus uniforme et on constate que la méthode du Gradient converge avec des oscillations irrégulières.

Ces résultats confirment que le choix optimal de relax est crucial pour garantir la

convergence et optimiser les performances des méthodes itératives.

Enfin, cette comparaison souligne la complexité des calculs sur des systèmes mals conditionnés. Le préconditionnement peut donc sembler très important pour améliorer la convergence de méthodes telle que celle du Gradient Conjugué. L'analyse des relaxations optimales et la gestion des grandes matrices restent également des pivots importants pour des simulations plus réalistes et efficaces.

3 Modes de vibration

Modes propres et fréquences propres d'une structure

Un **mode propre** est un mode d'oscillation caractéristique d'un système mécanique lorsqu'il est laissé libre d'évoluer sans excitation extérieure ni entretien des vibrations. Ces oscillations sont généralement amorties. Il correspond au mouvement simultané de tous les points du système à une **fréquence propre**, notée ω . Cette fréquence est une caractéristique de la structure, dépendant uniquement de ses caractéristiques physiques.

Le problème aux valeurs propres généralisé

Le problème aux valeurs propres généralisé vise à étendre le concept classique des valeurs et vecteurs propres en introduisant deux matrices, souvent notées A et B . L'objectif est de résoudre l'équation $Ax = \lambda Bx$, où λ désigne les **valeurs propres** et x les **vecteurs propres**. Cette approche permet de modéliser des systèmes complexes, notamment en mécanique et en dynamique des structures.

En mécanique, cette méthode joue un rôle central dans l'analyse des modes propres de structures soumises à des vibrations.

Le modèle étudié dans la première partie permet d'analyser les modes propres. Il s'agit de déterminer les solutions harmoniques dépendant du temps, exprimées sous la forme :

$$\underline{D}^i(t) = \underline{d}^i \cos(\omega t),$$

où ω est la fréquence propre et $\underline{d}^i$ le vecteur propre associé.

On a l'équation suivante pour chaque nœud i :

$$m_i \ddot{\underline{D}}_i = - \sum_j k_{ij} ((\underline{D}^i - \underline{D}^j) \cdot \underline{u}^{ij}) \underline{u}^{ij} \quad \text{avec} \quad \underline{u}^{ij} = \frac{\underline{p}^i - \underline{p}^j}{\|\underline{p}^i - \underline{p}^j\|_2} \in \mathbb{R}^3$$

On suppose que les déplacements prennent la forme :

$$\underline{D}^i(t) = \underline{d}^i \cos(\omega t),$$

d'où :

$$\ddot{\underline{D}}_i = -\underline{d}^i \omega^2 \cos(\omega t).$$

En remplaçant cette expression dans l'équation initiale, on obtient :

$$m_i \underline{d}^i \omega^2 \cos(\omega t) = - \sum_j k_{ij} ((\underline{d}^i \cos(\omega t) - \underline{d}^j \cos(\omega t)) \cdot \underline{u}^{ij}) \underline{u}^{ij}.$$

Ce qui peut être réécrit de manière équivalente comme suit :

$$m_i \underline{d}^i \omega^2 \cos(\omega t) = - \sum_j k_{ij} ((\underline{d}^i - \underline{d}^j) \cdot \underline{u}^{ij}) \underline{u}^{ij} \cos(\omega t).$$

On considère que $\cos(\omega t)$ n'est pas nul ($\omega t \neq \frac{\pi}{2} + k\pi$, où $\mathbb{K}$ est un entier), ce qui donne :

$$m_i \underline{d}^i \omega^2 = - \sum_j k_{ij} ((\underline{d}^i - \underline{d}^j) \cdot \underline{u}^{ij}) \underline{u}^{ij}.$$

Cela peut être réécrit sous forme matricielle comme suit :

$$m_i \underline{d}^i \omega^2 = \mathbb{K} \underline{d},$$

où $\mathbb{K}$ est la matrice de rigidité.

On définit la matrice de masse comme une matrice diagonale de taille $3n \times 3n$. Les éléments diagonaux de $\mathbb{M}$ sont les masses m_i des nœuds :

$$\mathbb{M}_{ij} = \begin{cases} m_i & \text{si } i = j, \\ 0 & \text{si } i \neq j. \end{cases}$$

Autrement dit, la matrice de masse $\mathbb{M}$ est de la forme suivante :

$$\mathbb{M} = \begin{bmatrix} m_1 & 0 & 0 & \cdots & \cdots & 0 \\ 0 & m_1 & 0 & \cdots & \cdots & 0 \\ 0 & 0 & m_1 & \cdots & \cdots & 0 \\ 0 & 0 & 0 & m_2 & \cdots & 0 \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & \cdots & m_n \end{bmatrix},$$

où chaque m_i est répété trois fois sur la diagonal pour chaque nœud i .

Ainsi, l'équation différentielle décrivant les modes propres s'écrit sous la forme matricielle suivante :

$$\mathbb{M}\underline{d}\omega^2 = \mathbb{K}\underline{d}.$$

Cela revient à résoudre le problème aux valeurs propres pour les matrices de masse $\mathbb{M}$ et de rigidité $\mathbb{K}$. Par identification avec l'équation du problème aux valeurs propres généralisé, on a :

$$\lambda = \omega^2.$$

D'autre part, en multipliant la relation trouvée auparavant par $\underline{d}^T$, on obtient :

$$\underline{d}^T \mathbb{M} \underline{d} \omega^2 = \underline{d}^T \mathbb{K} \underline{d}.$$

Ainsi, la relation suivante peut être établie pour ω^2 :

$$\omega^2 = \frac{\underline{d}^T \mathbb{K} \underline{d}}{\underline{d}^T \mathbb{M} \underline{d}}.$$

Propriété 1 : Soit $A \in \mathcal{M}_n(\mathbb{R})$ une matrice symétrique. Alors, chaque valeur propre de A est réelle.

Dans notre cas, la matrice $\mathbb{K}$ est symétrique semi-définie positive, et la matrice $\mathbb{M}$ est diagonale. Par conséquent, $\mathbb{M}$ est également symétrique, et leurs valeurs propres sont réelles.

D'autre part, $\mathbb{M}$ est définie positive, pour un vecteur quelconque $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, on a :

$$\mathbf{x}^T \mathbb{M} \mathbf{x} = \sum_{i=1}^n \sum_{j=1}^n \mathbb{M}_{ij} x_j x_i.$$

Comme $\mathbb{M}$ est diagonale ($\mathbb{M}_{ij} = 0$ pour $i \neq j$), cette double somme se réduit à une seule somme sur la diagonale :

$$\mathbf{x}^T \mathbb{M} \mathbf{x} = \sum_{i=1}^n m_i x_i^2.$$

- Les masses m_i sont positives ($m_i > 0$).
- $x_i^2 \geq 0$ pour tout i .

D'où :

$$\mathbf{x}^T \mathbb{M} \mathbf{x} > 0 \quad \text{pour tout } \mathbf{x} \neq 0.$$

Par ailleurs, comme $\mathbb{K}$ est symétrique semi-définie positive, on a :

$$\lambda = \omega^2 = \frac{\underline{d}^T \mathbb{K} \underline{d}}{\underline{d}^T \mathbb{M} \underline{d}} \geq 0.$$

Ainsi, les valeurs propres λ sont réelles et positives.

Propriété intéressante de la matrice $\mathbb{M}$

Soient $\underline{d}_1$ et $\underline{d}_2$ deux vecteurs propres distincts de la matrice $\mathbb{M}$, associés respectivement aux valeurs propres λ_1 et λ_2 , avec $\lambda_1 \neq \lambda_2$. Les équations caractéristiques sont données par :

$$\begin{cases} \mathbb{M} \underline{d}_1 = \lambda_1 \mathbb{K} \underline{d}_1, \\ \mathbb{M} \underline{d}_2 = \lambda_2 \mathbb{K} \underline{d}_2. \end{cases}$$

En multipliant la première équation à gauche par $\underline{d}_2^T$ et la seconde par $\underline{d}_1^T$, on obtient :

$$\begin{cases} \underline{d}_2^T \mathbb{M} \underline{d}_1 \lambda_1 = \underline{d}_2^T \mathbb{K} \underline{d}_1, \\ \underline{d}_1^T \mathbb{M} \underline{d}_2 \lambda_2 = \underline{d}_1^T \mathbb{K} \underline{d}_2. \end{cases}$$

Propriété 2 : Si $\mathbb{A}$ est une matrice symétrique de taille $n \times n$ (i.e., $\mathbb{A}_{ij} = \mathbb{A}_{ji}$), alors pour tous $\underline{x}, \underline{y} \in \mathbb{R}^n$, la propriété suivante est vérifiée :

$$\underline{x}^T \mathbb{A} \underline{y} = \underline{y}^T \mathbb{A} \underline{x}.$$

Démonstration : Développons l'expression $\underline{x}^T \mathbb{A} \underline{y}$:

$$\underline{x}^T \mathbb{A} \underline{y} = \sum_{i=1}^n \sum_{j=1}^n \mathbb{A}_{ij} x_j y_i.$$

En utilisant la symétrie de $\mathbb{A}$, où $\mathbb{A}_{ij} = \mathbb{A}_{ji}$, on peut écrire :

$$\sum_{i=1}^n \sum_{j=1}^n \mathbb{A}_{ij} x_j y_i = \sum_{i=1}^n \sum_{j=1}^n \mathbb{A}_{ji} x_j y_i.$$

En intervertissant les sommes, cela devient :

$$\sum_{i=1}^n \sum_{j=1}^n \mathbb{A}_{ij} x_j y_i = \sum_{j=1}^n \sum_{i=1}^n \mathbb{A}_{ji} x_j y_i.$$

Et enfin en intervertissant les indices i et j on obtient :

$$\sum_{i=1}^n \sum_{j=1}^n \mathbb{A}_{ji} x_j y_i = \sum_{i=1}^n \sum_{j=1}^n \mathbb{A}_{ij} y_j x_i = \underline{y}^T \mathbb{A} \underline{x}$$

Ainsi, on obtient :

$$\underline{x}^T \mathbb{A} \underline{y} = \underline{y}^T \mathbb{A} \underline{x}.$$

□

D'après cette propriété, puisque $\mathbb{K}$ est symétrique on a :

$$\underline{d}_1^T \mathbb{K} \underline{d}_2 = \underline{d}_2^T \mathbb{K} \underline{d}_1.$$

En utilisant cette propriété, nous réécrivons le système :

$$\begin{cases} \underline{d}_2^T \mathbb{M} \underline{d}_1 \lambda_1 = \underline{d}_1^T \mathbb{K} \underline{d}_2, \\ \underline{d}_1^T \mathbb{M} \underline{d}_2 \lambda_2 = \underline{d}_1^T \mathbb{K} \underline{d}_2. \end{cases}$$

Nous obtenons ensuite que :

$$\underline{d}_2^T \mathbb{M} \underline{d}_1 \lambda_1 = \underline{d}_1^T \mathbb{M} \underline{d}_2 \lambda_2.$$

Or, puisque $\mathbb{M}$ est symétrique, on a :

$$\underline{d}_2^T \mathbb{M} \underline{d}_1 = \underline{d}_1^T \mathbb{M} \underline{d}_2.$$

Ainsi, il en résulte :

$$\underline{d}_1^T \mathbb{M} \underline{d}_2 (\lambda_1 - \lambda_2) = 0.$$

Avec $\lambda_1 \neq \lambda_2$, nous concluons que :

$$\underline{d}_1^T \mathbb{M} \underline{d}_2 = 0.$$

Ce qui implique que :

$$\mathbb{M} \underline{d}_1 \cdot \underline{d}_2 = 0.$$

3.1 Importance des modes propres

Dans cette partie, on s'intéresse aux modes propres. Comme mentionné précédemment, ce sont des formes de vibration naturelles d'un système mécaniques qui correspondent à des fréquences propres spécifiques. Chaque mode propre est associé à une fréquence propre qui est la fréquence à laquelle le système tend à vibrer. Dans notre contexte, les modes propres décrivent les informations spécifiques possibles du ponts sous des conditions idéales où les déformations décrivent des mouvement verticaux, latéraux ou encore torsionnels. Si une fréquence d'excitation correspond à une fréquence propre du pont, cela peut entraîner une amplification des vibrations, donc une résonnance, qui peut causer des déformations importantes ou des dommages structurels. C'est pourquoi dans cette partie, on s'intéressera à ces derniers puisqu'ils jouent un rôle majeur dans la sécurité des ponts en aidant à anticiper les défaillances dues à des vibrations excessives ainsi que de concevoir des structures capables de supporter ces charges dynamiques.

Comme expliqué plus tôt, le problème de modes propres peut s'apparenter à un problème où l'on doit trouver les vecteurs et les valeurs propres associés au problème suivant :

$$Kd = \omega^2 Md$$

où les valeurs propres ω^2 correspondent aux carrés des fréquences propres du système et les vecteurs propres d décrivent les formes de déformation associées à chaque fréquence propre. Ainsi pour ce faire, on se propose d'utiliser un algorithme appliquant la méthode de puissance itérée inverse qui est utilisée pour calculer les valeurs propres ainsi que les vecteurs propres d'une matrice. Expliquons dès lors comment il fonctionne :

Dans un premier temps, une estimation initiale x_0 est donnée pour le vecteur propre lors de l'initialisation. Cette estimation est choisie arbitrairement mais elle doit vérifier le fait de ne pas être orthogonal au vecteur propre recherché puisque sinon ce dernier n'aura aucune composante dans x_0 . C'est une contrainte importante lorsque l'on utilise une méthode reposant sur des projections spectrales.

Ensuite, lors de la déflation, on supprime les composantes des vecteurs propres déjà calculés pour éviter qu'ils perturbent le calcul des nouveaux. En effet, lors des itérations précédentes, les vecteurs propres calculés sont projetés hors du vecteur actuel x_1 . L'étape de déflation est importante si l'on souhaite donc que l'algorithme ne retombe pas sur des vecteurs propres déjà trouvés et garantit que les nouveaux vecteurs propres calculés soient orthogonaux aux précédents afin que chaque mode propre représente un comportement indépendant du système.

On normalise après le vecteur x_1 afin d'éviter qu'il ne devienne trop grand ou bien alors trop petit au cours des itérations, c'est une normalisation intermédiaire.

On résout ensuite le système linéaire afin d'affiner l'estimation du vecteur propre. Ainsi à chaque itération, le système est résolu pour trouver une nouvelle approximation du vecteur propre y . Le système est de la forme :

$$y = \mathbb{K}^{-1} M x_0$$

où $\mathbb{K}$ est la matrice de raideur, M est la matrice de masse et x_0 est le vecteur avant mise à jour. Par ailleurs, pour utiliser $\mathbb{K}^{-1}$, on utilise la méthode QR puisque celle-ci est bien plus souvent efficace pour déterminer l'inverse de $\mathbb{K}$ sur des grandes matrices comparé à l'inverser directement. En fait, inverser $\mathbb{K}$ amplifie les composantes associées aux petites valeurs propres, donc celles qui correspondent aux modes dominants ou aux fréquences basses.

Pour finir, on réalise une dernière normalisation pour ajuster l'échelle du vecteur propre x_1 afin qu'il soit utilisable.

Aussi, tout au long de la méthode on vérifie si la méthode a convergé en vérifiant que l'er-

reur relative est bien inférieure à la tolérance et qu'on ne dépasse pas le nombre maximum d'itérations.

3.2 Tests et interprétations

Nous allons dès lors appliquer la méthode des modes propres au pont simple afin d'avoir un temps de calcul assez rapide et comprendre en quoi les valeurs des valeurs propres jouent un rôle sur l'intensité des déformations. Commençons ainsi en calculant 48 modes propres pour ainsi avoir 48 fréquences pour le pont simple. On trouve :

$$\lambda = \begin{bmatrix} 0.02568 & 0.02568 & 0.13595 & 0.13595 & 0.36448 & 0.42735 & 0.36448 & 0.42735 \\ 0.80347 & 1.00000 & 0.80347 & 1.00000 & 1.37729 & 1.42971 & 1.37729 & 1.42971 \\ 1.00000 & 2.06818 & 2.27947 & 1.00000 & 2.06818 & 2.27947 & 1.00000 & 2.00000 \\ 2.00000 & 2.00000 & 2.00000 & 3.26281 & 3.40838 & 4.69312 & 4.69312 & 3.26281 \\ 3.40838 & 4.69312 & 4.69312 & 1.00000 & 1.00000 & 0.02568 & 0.02568 & 0.02568 \\ 0.02568 & 0.13595 & 0.36448 & 0.13595 & 0.42735 & 0.36448 & 0.42735 & 0.42735 \end{bmatrix}$$

On a que les valeurs propres les plus petites correspondent à celles surlignées en bleu et que les valeurs propres les plus grandes correspondent à celle surlignée en rouge. Regardons dès lors pour quelques uns comment le pont se déforme pour ces dernières :

- Pour le **mode 0**, on remarque que le pont a une très grande déformation ; il fait des va et vient latéralement très importants
- Pour le **mode 1**, on remarque que le pont a aussi une très grande déformation, cependant bien que la valeur propre soit la même la, déformation n'est pas la même, le pont semble s'étirer puis revenir vers sa forme initiale.
- Pour le **mode 29** on remarque que la déformation est très moindre. On a simplement deux arcs qui semblent être déformés.
- Pour le **mode 34** on remarque le même phénomène que pour le mode 29, la déformation semble être très faible.

On remarque que plus la valeur propre est petite, plus la déformation est importante et au contraire plus la valeur propre est grande, plus la déformation est réduite. Ce phénomène peut s'expliquer par le fait que la résonance se produit si la fréquence de l'excitation, donc les vibrations, correspond à une des fréquences naturelles du pont. En fait, plus la valeur propre est grande, plus la fréquence naturelle est élevée ce qui signifie que des forces oscillant à des fréquences élevées seront nécessaires pour atteindre

la résonance. Par ailleurs, lorsque la résonance est atteinte, l'amplitude de la déformation est déterminée par la proximité de la fréquence d'excitation. Comme le pont est souvent soumis à des excitations périodiques comme le vent, les piétons, les véhicules ou encore les séismes, il est très important de bien choisir sa structure et ses matériaux pour éviter à tout prix que se dernier ne se déforme trop et qu'il s'effondre. On peut ainsi dire que la valeur propre détermine à quelle fréquence un mode propre est excité. Quand il y a résonance, plus la fréquence naturelle est basse et plus le mode est susceptible de causer une déformation importante car les systèmes mécaniques ont un amortissement faible, c'est-à-dire que les vibrations à basse fréquence, donc à basse valeur propre, persistent plus longtemps et atteignent donc des amplitudes plus élevées.

Désormais, nous allons tester les deux ponts que nous avons réalisés afin de voir à quels points ces derniers sont stables :

Pour le pont à treillis inversé :

$$\lambda_{treillis} = \begin{bmatrix} 0.03221 & 0.03817 & 0.04908 & 0.05715 & 0.11140 \\ 0.12475 & 0.18041 & 0.19632 & 0.20115 & 0.24697 \\ 0.30866 & 0.32440 & 0.34973 & 0.36810 & 0.39505 \\ 0.46016 & 0.47051 & 0.46235 & 0.48345 & 0.52810 \\ 0.56041 & 0.58537 & 0.67420 & 0.76827 & 0.79794 \\ 0.80680 & 0.88261 & 0.88931 & 0.94478 & 0.96514 \\ 0.97857 & 1.03689 & 1.04542 & 1.06214 & 1.21025 \\ 1.22667 & 1.21025 & 1.37457 & 1.41903 & 1.38350 \\ 1.46466 & 1.54686 & 1.54686 & 1.69491 & 1.79002 \\ 1.78408 & 1.82599 & 1.92564 & 1.82599 & 1.92564 \end{bmatrix}$$

Pour le pont de Sydney :

$$\lambda_{\text{Sydney}} = \begin{bmatrix} 1.19739e-10 & 9.67263e-05 & 2.21635e-05 & 1.06227e-10 & 0.00203 \\ 2.21635e-05 & 9.67263e-05 & 9.67263e-05 & 9.67263e-05 & 9.67263e-05 \\ 0.00899 & 2.21635e-05 & 4.37122e-10 & 0.01271 & -8.91914e-09 \\ 0.00203 & 0.02278 & 2.55227e-10 & 5.72382e-10 & 2.21635e-05 \\ 3.91280e-10 & 0.04355 & 2.21635e-05 & 2.21635e-05 & 9.67263e-05 \\ 4.27008e-10 & 1.01160e-10 & 0.07219 & 0.00053 & 9.67263e-05 \\ 1.92789e-11 & 9.67263e-05 & 2.53258e-09 & 6.33709e-10 & 6.40424e-10 \\ 5.92328e-10 & 2.48413e-10 & 1.14443e-10 & 9.67263e-05 & 9.67263e-05 \\ 2.21635e-05 & 2.21635e-05 & -7.08480e-11 & 1.00000e-10 & 2.21635e-05 \\ 0.17717 & 9.96171e-11 & 9.67263e-05 & & \end{bmatrix}$$

On remarque que les ponts que nous avons réalisés sont bien plus sensibles aux résonances. En effet, les valeurs propres les plus hautes pour les deux ponts sont bien plus petites que celles du pont simple et les valeurs propres les plus basses sont encore bien plus petites que celles du pont simple. Bien que spatialement correct, nos ponts semblent être bien trop simplistes et bien trop peu réalistes mais nous ont cependant permis de mettre en exergue les phénomènes de déformations que ce soit par la pression uniforme, le cisaillement ou ici la résonance avec les modes propres. Par ailleurs, il est très intéressant de remarquer que le conditionnement de la matrice joue un rôle très important dans l'apparition de valeurs propres très basses. En effet, le conditionnement de la matrice $\mathbb{K}$ pour le pont de Sydney était de l'ordre de e^{17} et on remarque qu'on a des valeurs propres de l'ordre de e^{-10} (par ex $1.19739e^{-10}$). Les valeurs propres proches de zéro signifie que certaines directions sont presque singulières et donc la matrice a des directions privilégiées dans lesquelles elle agit de manière quasi neutre.

3.3 Affinage et valeurs propres

Pour pouvoir améliorer de manière successive les valeurs propres ainsi que les vecteurs propres du système, on utilise des itérations supplémentaires pour affiner la solution. L'affinage vise à calculer de manière plus précise les valeurs propres et les vecteurs propres associés à la matrice $\mathbb{K}$. Il est utilisé après une première estimation grossière des modes propres. Il permet ainsi d'améliorer la précision de la solution. L'affinage est utile dans les cas où les estimations initiales des valeurs ainsi que des vecteurs propres ne sont pas assez précises. Après avoir effectué un premier calcul l'affinage permet de corriger les erreurs et de rendre les résultats plus précis notamment en ce qui concerne les petites variations

dans les valeurs et vecteurs propres.

Pour ce faire, l'algorithme calcule les valeurs et vecteurs propres pour un système perturbé par la matrice $\mathbb{K}$ moins un petit ajustement qui est $(LL[i] - 1e - 10).D$ où LL représente la matrice des valeurs propres. L'objectif ici est donc d'affiner les valeurs et vecteurs propres en ajustant les estimations avec une précision plus fine. Par ailleurs, la tolérance, définie ici par $1e - 10$ va aussi avoir un impact sur la précision et le temps des calculs. Il est donc important de bien choisir ce paramètre.

L'affinage est donc une méthode pour améliorer progressivement les résultats des calculs de valeurs et de vecteurs propres en utilisant des itérations supplémentaires ce qui est un très grand enjeu dans des simulations de systèmes dynamiques comme les structures de ponts.

4 Conclusion

Lors de ce travail pratique sur les méthodes itératives et la déformation de différents ponts, nous avons approfondi notre compréhension des phénomènes de déformation des structures. En particulier, nous avons modélisé et appliqué les différentes méthodes itératives vues en cours d'analyse numérique afin de mieux comprendre leurs capacités et leurs limites. Par ailleurs, nous avons étudié le phénomène de résonance en travaillant sur les modes propres des ponts. Ce travail pratique nous a également permis de saisir l'importance des méthodes itératives dans la résolution efficace de systèmes linéaires complexes, un enjeu majeur dans la conception de ponts. Enfin, en abordant de manière simplifiée une problématique courante du génie civil, nous avons démontré que ces méthodes, bien que simples dans leur principe, offrent une flexibilité remarquable en termes de précision, permettant d'analyser des structures toujours plus complexes tout en garantissant leur stabilité et leur sécurité. Elles jouent ainsi un rôle fondamental dans l'optimisation des conceptions et dans l'anticipation des comportements dynamiques des structures.

1 Annexe

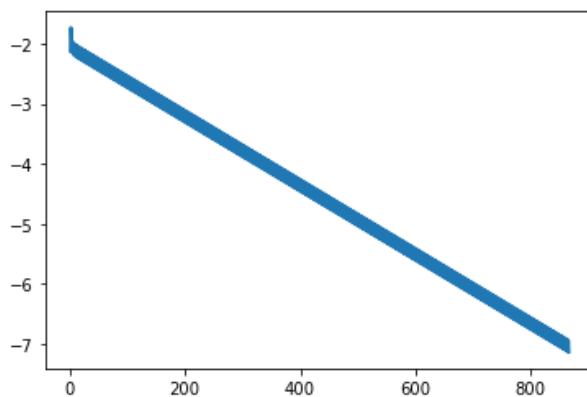


FIGURE 4 – Évolution logarithmique des résidus au cours des itérations pour $tol = 1e - 5$, $itemax = 1000$, $relax = 1$

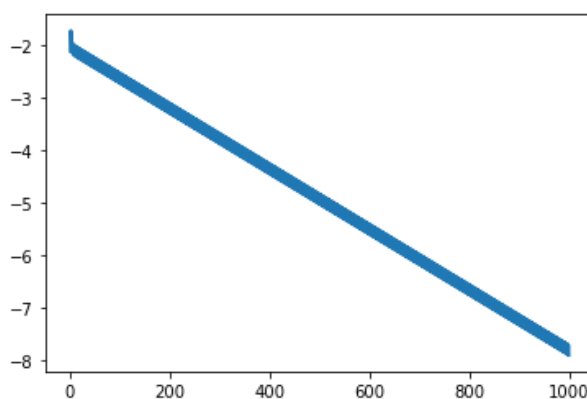


FIGURE 5 – Évolution logarithmique des résidus au cours des itérations pour $tol = 1e - 8$, $itemax = 1000$, $relax = 1$

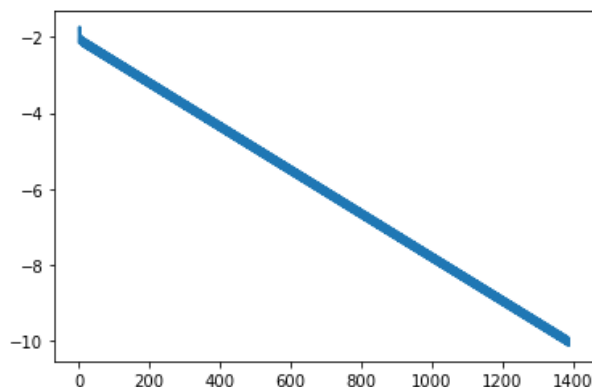


FIGURE 6 – Évolution logarithmique des résidus au cours des itérations pour $tol = 1e - 8$, $itemax = 2000$, $relax = 1$

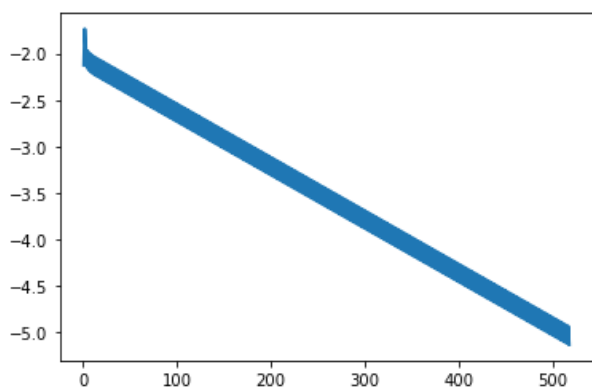


FIGURE 7 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e - 3$, $itemax = 1000$, $relax = 1$

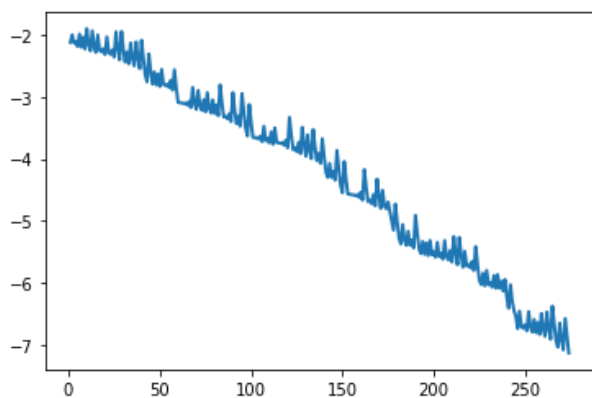


FIGURE 8 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e - 5$, $itemax = 1000$, $relax = 0.5$

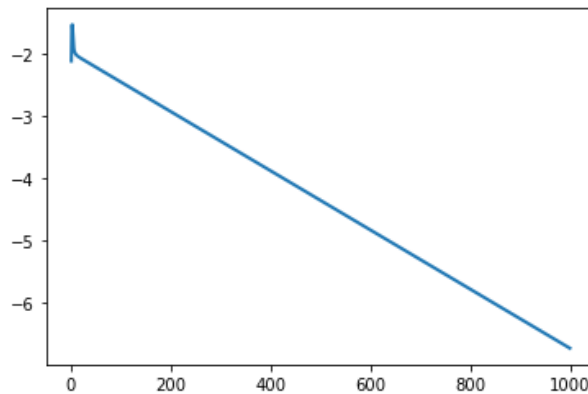


FIGURE 9 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-5$, $itemax = 1000$, $relax = 1.5$

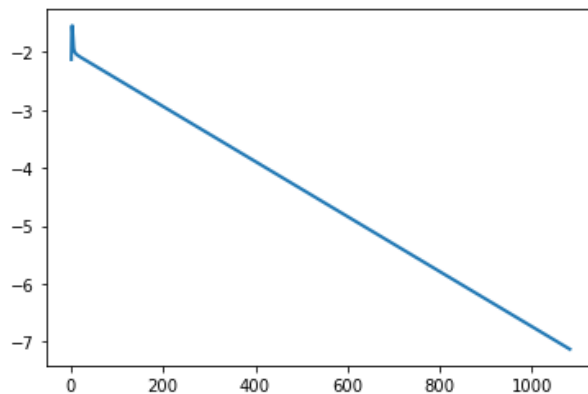


FIGURE 10 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-5$, $itemax = 2000$, $relax = 1.5$

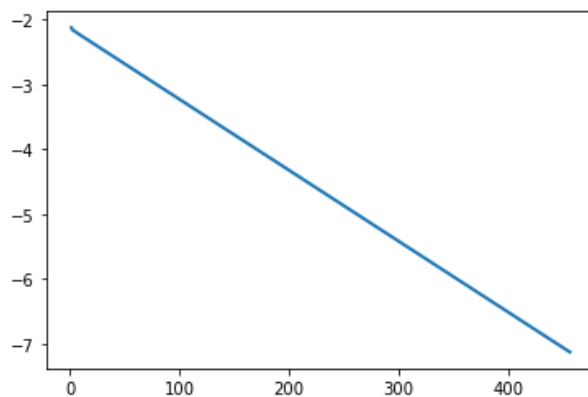


FIGURE 11 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-5$, $itemax = 1000$, $relax = 1$

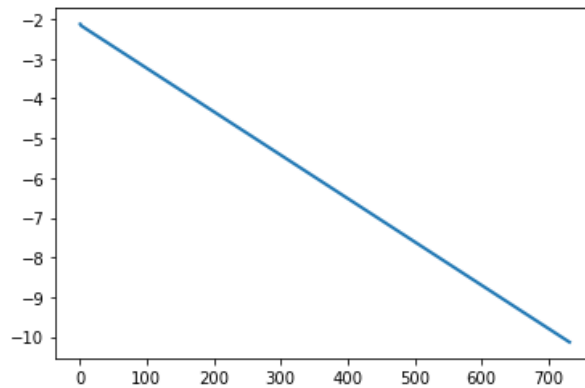


FIGURE 12 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-8$, $itemax = 1000$, $relax = 1$

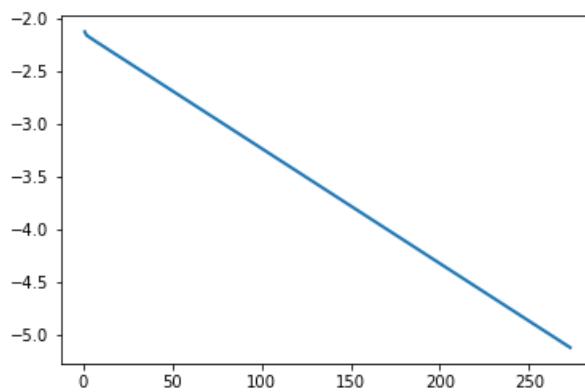


FIGURE 13 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-3$, $itemax = 1000$, $relax = 1$

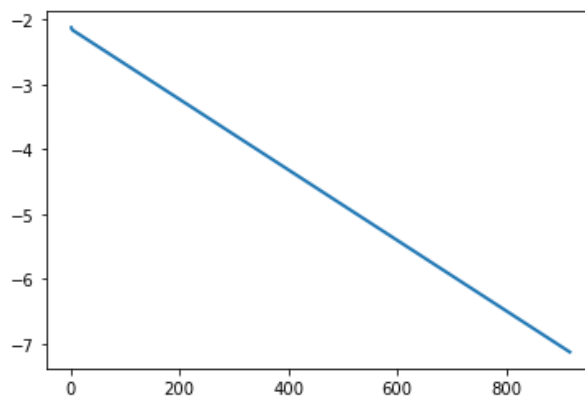


FIGURE 14 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-5$, $itemax = 1000$, $relax = 0.5$

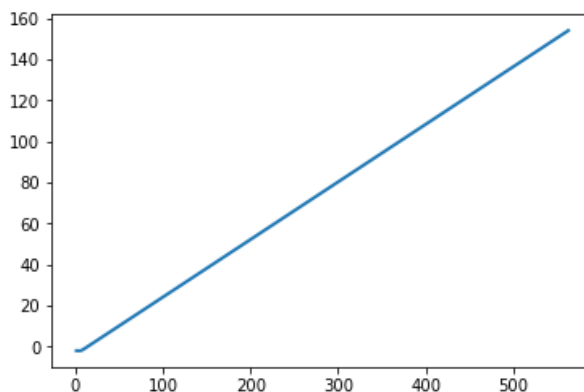


FIGURE 15 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-5$, $itemax = 1000$, $relax = 1.5$

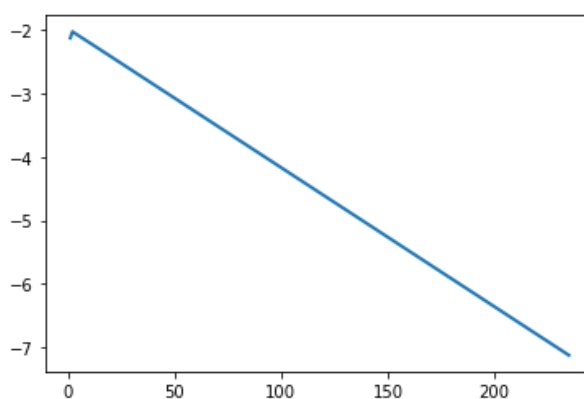


FIGURE 16 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-5$, $itemax = 1000$, $relax = 1$

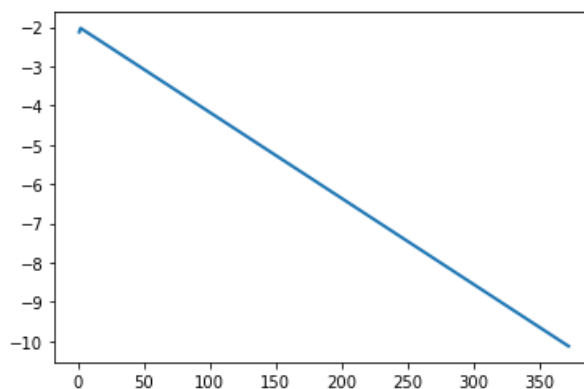


FIGURE 17 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-8$, $itemax = 1000$, $relax = 1$

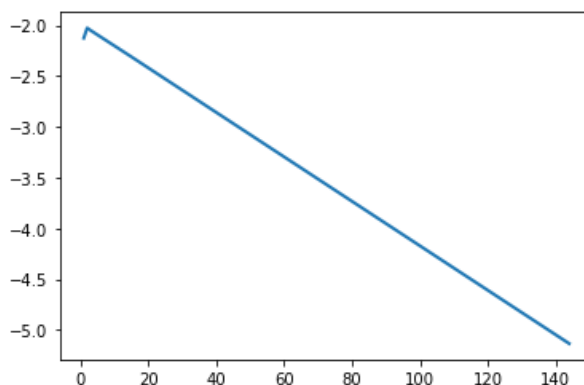


FIGURE 18 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-3$, $itemax = 1000$, $relax = 1$

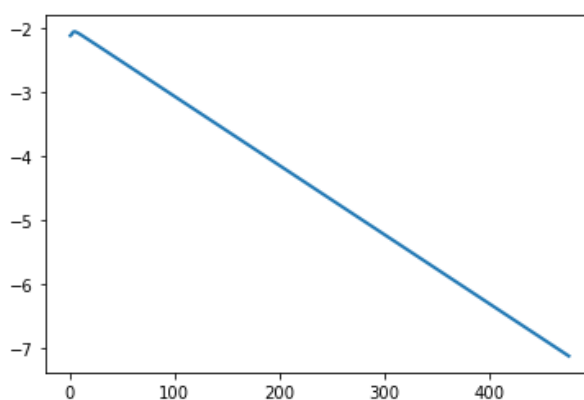


FIGURE 19 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-5$, $itemax = 1000$, $relax = 0.5$

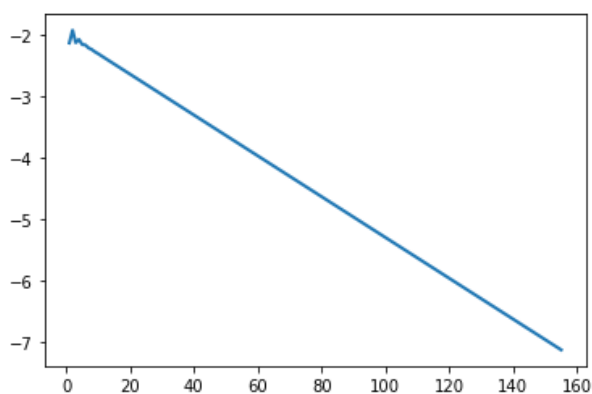


FIGURE 20 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-5$, $itemax = 1000$, $relax = 1.5$

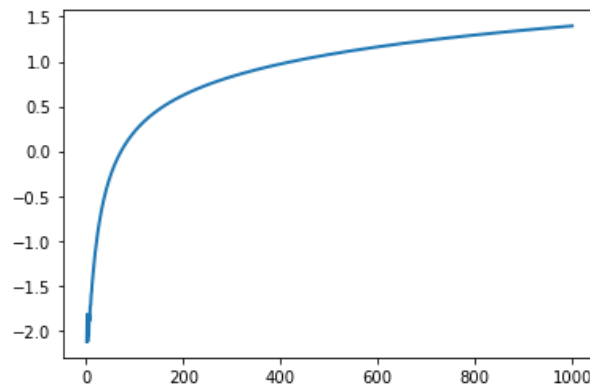


FIGURE 21 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-5$, $itemax = 1000$, $relax = 2.0$

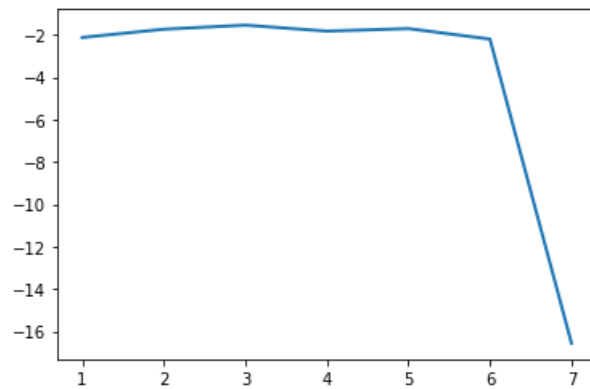


FIGURE 22 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-3$, $itemax = 1000$, $relax = 1$

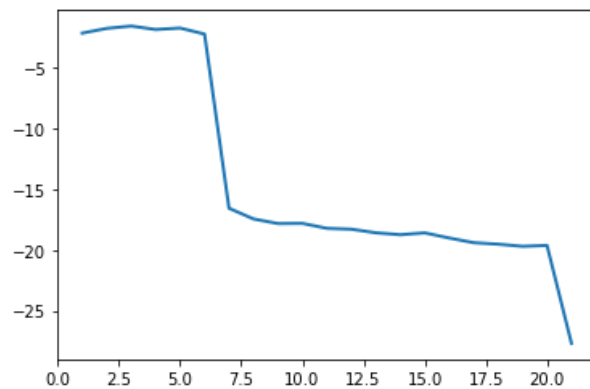


FIGURE 23 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e-18$, $itemax = 1000$, $relax = 1$

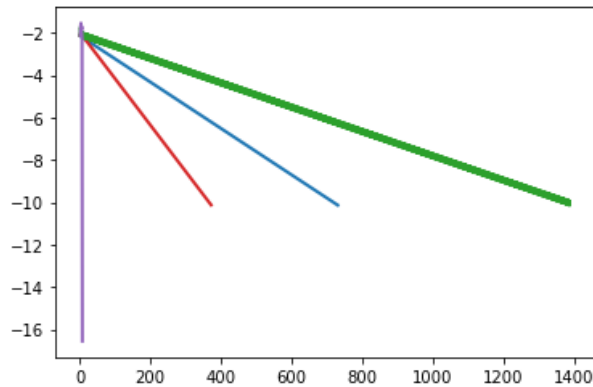


FIGURE 24 – BLEU=JACOBI,ROUGE=GAUSS-SEIDEL,VERT=GRADIENT, VIOLET=GRADIENT CONJUGUE, pression uniforme

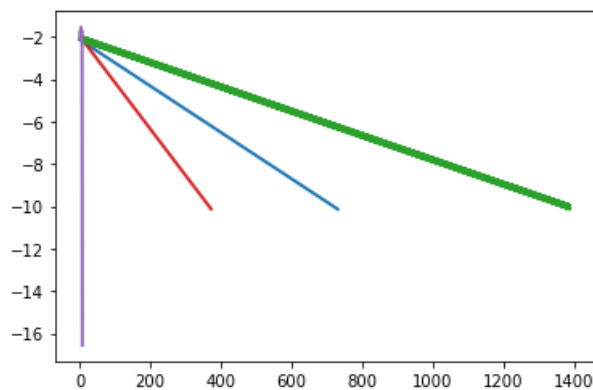


FIGURE 25 – BLEU=JACOBI,ROUGE=GAUSS-SEIDEL,VERT=GRADIENT, VIOLET=GRADIENT CONJUGUE, cisaillement

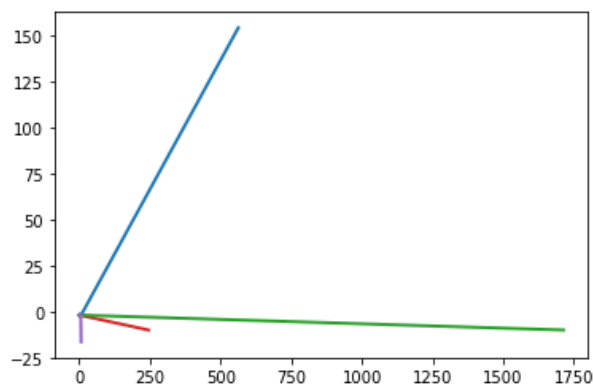


FIGURE 26 – BLEU=JACOBI,ROUGE=GAUSS-SEIDEL,VERT=GRADIENT, VIOLET=GRADIENT CONJUGUE, pression uniforme, $relax = 1.5$

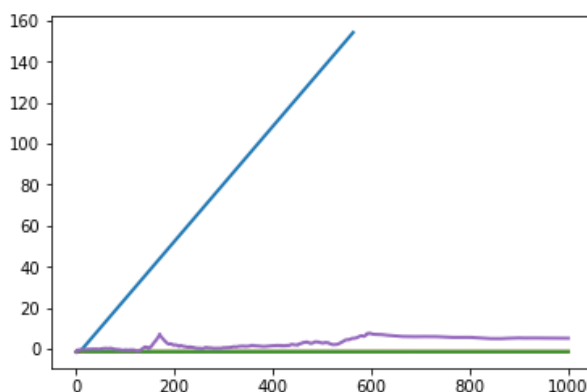


FIGURE 27 – Évolution logarithmique des résidus au cours des itérations, pour $tol = 1e - 5$, $itemax = 1000$, $relax = 1.0$

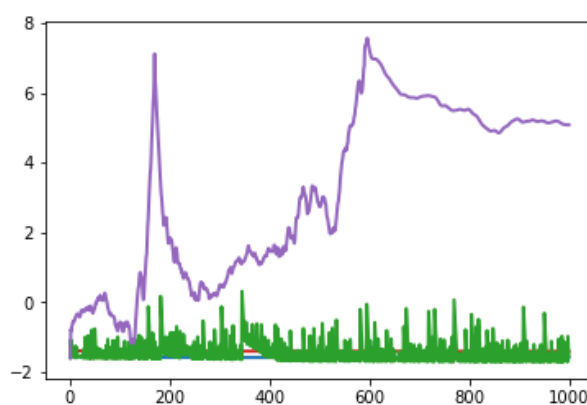


FIGURE 28 – Évolution logarithmique des résidus au cours des itérations, $tol = 1e - 5$, $itemax = 1000$, $relax = 0.5$

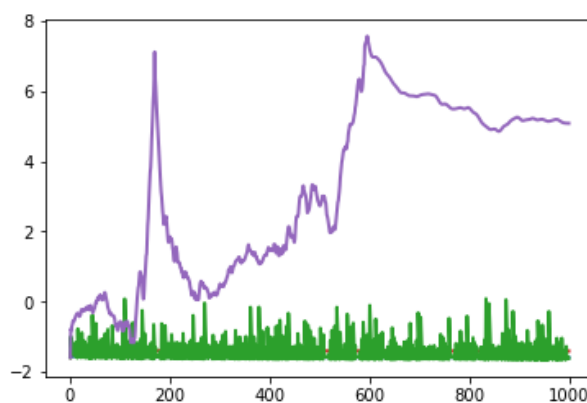


FIGURE 29 – Évolution logarithmique des résidus au cours des itérations, $tol = 1e - 5$, $itemax = 1000$, $relax = 0.6$

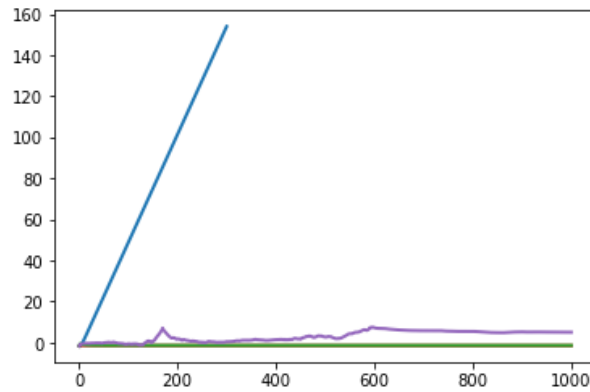


FIGURE 30 – Évolution logarithmique des résidus au cours des itérations, $tol = 1e - 5$, $itemax = 1000$, $relax = 1.5$

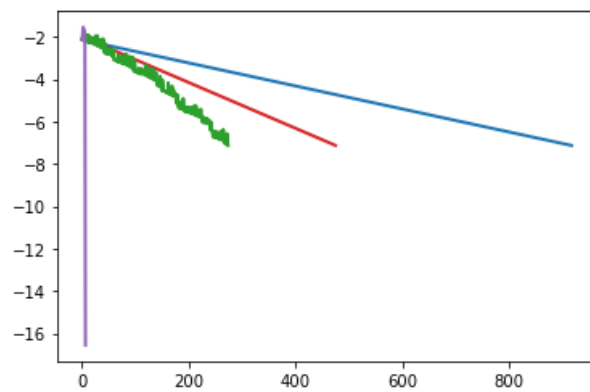


FIGURE 31 – Évolution logarithmique des résidus au cours des itérations, $relax = 0.5$

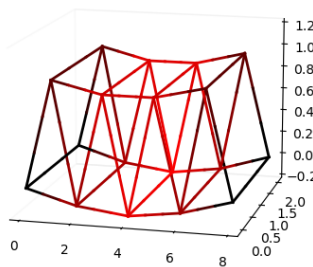


FIGURE 32 – Déformation du pont simple pour le mode 0

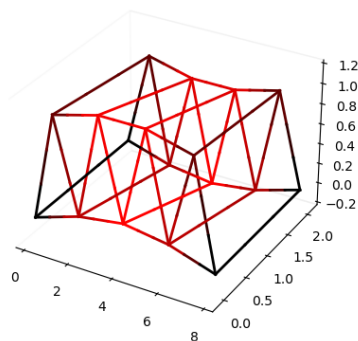


FIGURE 33 – Déformation du pont simple pour le mode 1

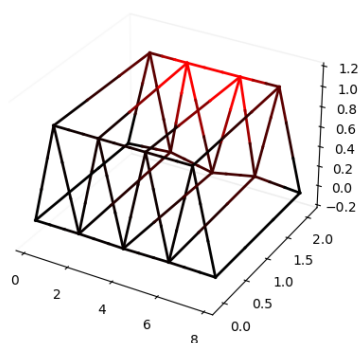


FIGURE 34 – Déformation du pont simple pour le mode 29

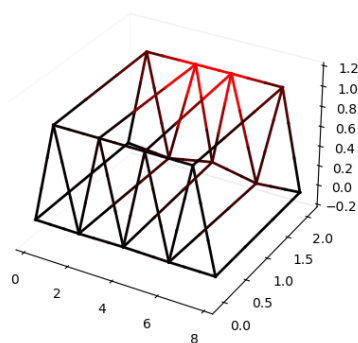


FIGURE 35 – Déformation du pont simple pour le mode 34

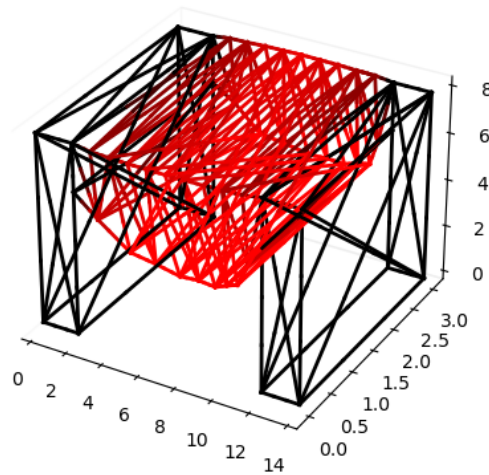


FIGURE 36 – Déformation du pont en treillis avec valeur propre petite

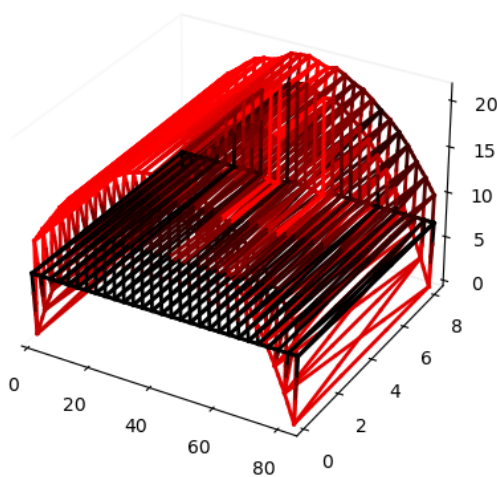


FIGURE 37 – Déformation du pont de Sydney avec valeur propre petite

Bibliographie

- [1] Pont de Sydney https://fr.wikipedia.org/wiki/Harbour_Bridge
- [2] Pont en treillis https://fr.wikipedia.org/wiki/Pont_en_treillis
- [3] Vibration des systèmes couplés https://maths.cnam.fr/IMG/pdf/cours_CSC216_Chap1.pdf
- [4] Théorie spectrale https://images-archive.math.cnrs.fr/Spectre.html?id_forum=7422